Brain2V

Full Skill Manual — every /obs-* command, explained and demonstrated

Repository: github.com/CYBERSAREEN/Brain2V
Generated: 2026-07-06
23 skills documented — no page limit, nothing abridged

Table of Contents

What Brain2V Is

Brain2V is a second-brain skill system for Claude Code, built on top of an Obsidian vault. It is not an application or a background service — there is no process running between your turns. Everything here is a **protocol layer**: skill definitions (`commands/`), shared conventions (`knowledge/`), and hook nudges (`hooks/`) that Claude Code follows while you work.

Three ideas run through every skill in this manual:

- **Fast lookup, not scanning.** Every note is registered in a two-layer index — a hash map (`date|time|day|label → note`) for O(1) exact lookup, and a symlink tree (`by-date/` , `by-day/` , `by-label/`) so "everything from Friday" or "everything tagged pentest" is a plain filesystem listing, as fast as the OS finds a file, regardless of how large the vault grows.
- **One graph, not silos.** Every note a skill writes cross-links to same-day and subject-related notes and carries a shared `#obs` tag, so the vault reads as one connected brain in Obsidian's own graph and backlinks view.
- **Store what's required, not everything.** Large sources (a PDF, an over-explained brief) get distilled to their essence plus a pointer to the original — never dumped raw — so every future read costs a fraction of the tokens with nothing actually lost.

This manual documents all 23 skills currently in the family: what each one does, why it exists, exactly how to invoke it, and a worked example grounded in its real behavior — not aspirational copy. Wherever a skill has a genuine limitation (something it cannot actually do, or an approximation it makes), that limitation is stated plainly in a boxed **Honest limit** note rather than glossed over. This is a deliberate house style across the whole project: the system is more useful when its guarantees are accurate.

1. /obs-organiser Orchestrator

The orchestrator ("brain") of the /obs- system — routes every request to the right skill, initializes the fast index, and drives the whole family as one coherent system.*

What it does

`/obs-organiser` is the routing-and-decision layer that ties every other skill in this manual together. On first invocation in a session it initializes cheaply — it reads the most recent Journal entry, the most recent context snapshot, and a directory listing of `.obs-index/by-label/` (a filesystem `ls`, not a vault scan) — enough situational awareness to route intelligently without reading the whole vault. From there, every request that plausibly maps to an obs skill gets routed: a fetch ("utha vedant personality") resolves via a hash lookup or a label-directory listing instead of a search; an action (logging a mistake, starting a pentest, connecting two topics) gets matched against the trigger table and either invoked or flagged to the user as the applicable skill.

The organiser also decides, for each candidate skill, whether to trigger it automatically or ask first. Low-risk, additive actions (logging a learning, updating the index, briefing at session start) trigger automatically. Anything that writes credentials (`/obs-personality`), runs active tooling against a target (`/obs-pentest`), or is genuinely ambiguous gets asked about first, one line, so the user can override.

Finally, the organiser owns the "employee-style" loop the whole system is built around: request → route → execute until the goal is met → learnings/mistakes captured (via the auto-trigger hooks) → human feedback → corrections written back. At the end of a unit of work, its job is to check the loop actually closed — was a mistake logged with its fix, was a reusable lesson saved — and close it if not.

Standing duty: keeping Brain2V in sync

The organiser also carries a pre-authorized standing responsibility: whenever any `/obs-*` command file, shared protocol, or hook script changes, it syncs the change into the Brain2V repository, secret-scans the diff, commits, and pushes — without asking permission for the push itself each time, though it always reports afterward what was pushed and to which commit.

> **Honest limit:** the organiser is a protocol in force for the session, re-invoked at defined trigger points — not an always-on daemon "watching side by side." A hook or skill cannot run continuously between turns; nothing here polls in the background. Likewise, "faster on the second project" is not self-optimization — it is a direct consequence of resolving context through the index and reusing persisted work, and it only appears when that prior work genuinely exists to reuse.

Usage

Argument	Effect
`(none)`	Initializes for the session and waits to route the next request.
`<a request or goal>`	Routes that request directly, e.g. a fetch or an action mapped to a specific skill.

Example

```
/obs-organiser utha vedant personality
```

The organiser recognizes this as a fetch for a persona. Instead of searching the vault, it computes (or looks up) the hash key for `label:personality,` `title:vedant`, or lists `.obs-index/by-label/personality/`, resolves the symlink to `Personalities/vedant.md`, and reads that single note directly.

Related: every other skill in this manual is a candidate for organiser routing — see each skill's own section for its specific trigger condition.

2. /obs-guide Session Lifecycle

Reads back your recent journal/session/engagement data and briefs you on what's complete, what's incomplete, and what to start with next.

What it does

`/obs-guide` is the "what should I do right now" briefing. It pulls from every place other obs skills leave state — the last 2-3 `Journal/` entries (with special weight on the "Unfinished / carries to tomorrow" section, the single strongest incomplete-work signal in the vault), the most recent `Context/session-log.md` snapshot, any `Pentest/<target>/pen-context.md` engagement missing a final report, and recent `Mistakes/` / `Learnings/` entries whose `do-not` warning is relevant to whatever comes next.

It reconciles these into three honest buckets — complete (something later explicitly marked it done), incomplete (appears in a carryover list, nothing later closed it), and stale (present once, never mentioned again, not marked either way) — and ranks a single "Start here" recommendation rather than a flat list: an explicit carryover from the most recent Journal note outranks an open Context thread, which outranks an unfinished pentest engagement, which outranks anything else incomplete (oldest first).

Usage

Argument	Effect
`(none)`	Covers everything found across the vault.
`<project/focus>`	Narrows the briefing to that project or focus only, e.g. "excelonCS" or "pentest".

Example

```
/obs-guide
```

Produces a report headed by a single "Start here" line with its source note linked, then Complete / Incomplete / Stale / Relevant warnings sections — each saying "none found" explicitly rather than being silently omitted if empty.

> **Honest limit:** this is a live status read, not a saved artifact — a cached copy of "what to do next" from days ago would actively mislead, so it is deliberately never persisted to the vault.

Related: fires automatically via the `SessionStart` hook. Reads state left by /obs-closeday, /obs-retain-context, /obs-pentest, /obs-mistakes, /obs-learn.

3. /obs-closeday Session Lifecycle

Reviews what you worked on today and logs a dated close-of-day note to the vault — progress, new ideas, unfinished carryover. Counterpart to /obs-context-code.

What it does

At the end of a working day, `/obs-closeday` gathers what changed — notes modified today (via filesystem mtime), a summary of the session's own activity (files touched, decisions made, commands run — visible only from this session's own context, not stored anywhere else), and anything logged today by other obs skills — and writes it to `Journal/<YYYY-MM-DD>.md` under three sections: Progress, New ideas that came up, and Unfinished / carries to tomorrow. If the day's file already exists (re-running the same day), it merges new findings into the existing sections rather than duplicating them.

Per the linking protocol, this note is the day's hub: it links out to every other same-day note from any obs skill, and — since Obsidian backlinks are automatic — every one of those notes linking back here (per their own Related sections) means the day becomes a natural two-way entry point into everything that happened on it.

Usage

Argument	Effect
`(none)`	Only invocation form — no parameters.

Example

```
/obs-closeday
```

Writes or merges into `Journal/2026-07-06.md`, linking every note created today by `/obs-learn`, `/obs-mistakes`, `/obs-pentest`, etc.

Related: fires automatically via the `SessionEnd` hook (alongside `/obs-requests`). Read first by `/obs-guide`.

4. /obs-retain-context Session Lifecycle

Manually snapshots the full current session (in-flight work, decisions, open threads) into the vault with a timestamp — a save-point for when a session is running low on context room.

What it does

Builds a snapshot of the session as of right now — in-flight tasks, decisions made and why (especially anything non-obvious that isn't recoverable by re-reading code), files touched, and open threads — and appends (never overwrites) an entry to `Context/session-log.md` with an exact timestamp.

> **Honest limit, stated up front in the skill itself:** a slash command cannot fire itself automatically at a context-usage percentage — nothing invokes a skill without the user or a hook doing so. True automatic triggering requires a hook. This command is the manual save-point itself.

In practice, this limit is addressed by a real `PreCompact` hook wired in settings.json, which nudges Claude to run this command right before context is compacted — the hook provides the trigger, this command provides the save.

Usage

Argument	Effect
`(none)`	Only invocation form.

Example

```
/obs-retain-context
```

Appends a dated entry like:

```
## 2026-07-06 01:05 — Context snapshot
**In flight:** ...
**Decisions:** ...
**Files touched:** ...
**Open threads:** ...
**Related:** [[Journal/2026-07-06]]
```

Related: triggered by the `PreCompact` hook. Indexed once as a whole (`kind: context-log`), not once per entry, since it is an append-only log.

5. /obs-context-code Session Lifecycle

Loads your full life and work state into Claude Code — active projects, preferences, priorities, and current focus — by reading your Obsidian vault. Use at the start of any session where you want the agent to already know everything relevant about you.

What it does

Scans across every registered vault (not just the default) for: recently modified notes (last 7 days), journal/daily entries from that window, notes tagged `#project` / `#active` / `#priority` / `#focus` or carrying `status: active`, and standing preferences that aren't time-boxed. It organizes these into four buckets — Active projects, Recent reflections, Priorities mentioned, and Standing preferences — and reports each explicitly, saying "none found" rather than omitting an empty bucket.

Usage

Argument	Effect
`(none)`	Covers everything.
`<focus area>`	Narrows to that area, e.g. "work" or "personal".

Example

```
/obs-context-code work
```

Honest limit: like /obs-guide, this is a live session load, not a new artifact — it deliberately does not save a copy to the vault, since it is simply a current read of what already lives there.

Related: counterpart to /obs-closeday (which closes the day this one opens).

6. /obs-understanding Session Lifecycle

Builds Claude's understanding of the user's personal workflow, methodology, and philosophy — how they work and how they want Claude to work — by reading their Obsidian vault and existing memory.

What it does

This skill has a standing proactive trigger: it should be invoked via the Skill tool whenever "Obsidian" is mentioned in any form anywhere in the conversation — the vault, vault notes, or any obs-* command — not only when typed explicitly. The full search runs once per session on first mention; later mentions in the same session just apply what was already learned rather than re-running the search.

Usage

Argument	Effect
`(none)`	Only invocation form — but see the proactive trigger above.

Example

The user says "let's use Obsidian for this" mid-conversation, unrelated to any explicit slash command. This skill fires proactively, runs its search once, and folds the result into how the rest of the session treats the user's working style.

Related: feeds context that `/obs-code-personality` and `/obs-life` later formalize into standing, structured preferences.

7. /obs-learn Knowledge Capture

Captures something you just learned into the vault with the current date/time — general knowledge capture. For errors/mistakes specifically, use /obs-mistakes instead.

What it does

Appends a dated, titled entry to `Learnings/learnings.md` — one growing file, so the index maps topic → file rather than topic → per-entry path. Before appending, it checks for a near-duplicate entry on the same topic from the last ~30 days; if found, it appends a dated follow-up under the existing entry instead of creating a near-duplicate new one. Every entry includes a `related: [[Journal/<today>]]` line so the day's Journal note backlinks here automatically.

Usage

Argument	Effect
`<what you learned>`	Required — the skill asks and stops if empty rather than guessing.

Example

```
/obs-learn Vercel builds are case-sensitive on Linux even though macOS/Windows aren't
```

Produces an entry titled with a short derived heading, the learning in full, inferred tags, and the Journal backlink — filed into `Learnings/learnings.md`.

Related: distinct from /obs-mistakes (errors specifically) and /obs-learn-cyber (cybersecurity + AI domain specifically).

8. /obs-mistakes Knowledge Capture

Logs a mistake — build failure, wrong approach, or an AI hallucination/false-positive/false-negative — into the vault so it's never repeated. The Obsidian-side counterpart to ~/.claude/knowledge/learnings.md.

What it does

Classifies the mistake (build-failure, ai-hallucination, false-positive, false-negative, process-mistake, other) and appends a structured entry to `Mistakes/mistakes.md`, using the same entry format as `~/.claude/knowledge/learnings.md` so both stores read consistently: what happened, root cause, fix/correction, and — critically — a `do-not` line naming the specific wrong approach to never retry. If this exact mistake was logged before, that repetition is itself flagged explicitly (the earlier do-not clearly didn't stick). Code/build/deploy/security-related mistakes are also cross-written to `~/.claude/knowledge/learnings.md`, so the orchestrator pipeline's mandatory before-work read picks them up too.

Usage

Argument	Effect
`<what went wrong>`	Required.

Example

```
/obs-mistakes a migration script hashed a placeholder time shared across
three entries, causing a silent key collision that dropped two of them
```

Real example from this project's own history: classified as `process-mistake`, root cause and fix recorded, cross-written to `learnings.md`, tagged `hash-collision, obs-index, obs-list, backfill, indexing`.

Auto-trigger

A `PostToolUseFailure` hook nudges toward this skill whenever a tool call fails, once the failure is understood and fixed — logging happens after resolution, not at the moment of failure itself.

Related: feeds `~/.claude/knowledge/learnings.md` for code-related entries; distinct from the general-purpose / obs-learn.

9. /obs-learn-cyber Knowledge Capture

Learns cybersecurity + AI knowledge alongside you — PDFs, course/book notes, MCP concepts, or "what I just did" — acknowledges understanding back to you, and files it so /trace can show how your understanding evolved.

What it does

This is a domain-specific learning companion, distinct from the general-purpose `/obs-learn`. It classifies incoming material into one of four recognized categories — a PDF or file path (read directly, the Read tool handles PDFs natively), a course/book excerpt, an MCP-protocol concept (deliberately tied back to concrete MCP connections already live in the environment, such as the Burp Suite or Obsidian MCP servers), or a "what I did" activity log (extracted from the user's own description, not a document).

Critically, it **acknowledges its understanding back to the user in the chat reply before filing anything** — restating the core concept and specifically how it's framing the cybersecurity↔AI connection, flagging explicitly if any part is uncertain rather than filing a confident-sounding note built on a guess. This acknowledgment step is treated as the skill's actual job, with saving as the secondary persistence step.

Each learning event is filed as its **own dated note** — deliberately not appended to one growing log — specifically so that `/trace <topic>` can later show real evolution across sessions once a topic has two or more notes. A `CyberAI-index.md` hub tracks MCP concepts, sources, an activity log, and a curated list of intersection highlights.

Usage

Argument	Effect
`<pasted text>`	Course/book excerpt or general text.
`<path to PDF/file>`	Read in full and distilled into concepts.
`<a topic>`	e.g. "MCP server security" — logged as an MCP-concept entry.

Example

```
/obs-learn-cyber /home/kali/Downloads/ai-red-teaming-2026.pdf
```

Reads the PDF, restates in chat what was understood and how it connects cybersecurity and AI, then files `CyberAI/2026-07-06-ai-red-teaming.md` with source-type `pdf`, the extracted essence, and a Related link back to the index and today's Journal.

Related: designed explicitly to work with /trace. Distinct from /obs-learn (general) and /obs-mistakes (errors).

10. /obs-distil Knowledge Capture

"Remember this, minimally." Takes an over-explained brief, a whole PDF, or a long dump and stores ONLY the required essence in Obsidian (with a pointer to the source), so every future read costs far fewer tokens. Brain2V's native context-reduction feature.

What it does

This is Brain2V's storage-side token-efficiency feature. The source is read in full (a PDF, a pasted brief, an existing bloated note), but only the required essence is stored — decisions, specs, exact values, constraints, gotchas, acceptance criteria — and the note keeps a pointer to the untouched original rather than the raw material itself. The success test the skill applies: "would future work still have everything it needs from this shorter note, without re-reading the original?" If unsure whether a detail is load-bearing, it keeps it (or relies on the source pointer), rather than risk losing something genuinely needed.

Every capture skill in the family (`/obs-learn` , `/obs-learn-cyber` , `/obs-mistakes` , the automation-tool skills, `/obs-code-personality`) is expected to distil by default — none should paste a raw dump into the vault.

Usage

Argument	Effect
`<path to a PDF/file>`	Reads and distils it.
`<pasted text>`	Distils the pasted material directly.
`distil <existing note>`	Shrinks an already-bloated note in place.

Example

```
/obs-distil /home/kali/Documents/pentest-automation-whitepaper.pdf
```

Reports back something like: "distilled a 40-page PDF to a ~20-line note; original preserved at <path>" — with the note saved to `Distilled/2026-07-06-pentest-automation.md` .

> **Honest limit:** distillation is lossy on purpose — it keeps what's judged required. If the user explicitly wants the whole thing verbatim, the skill honours that, but says plainly it costs more tokens on every future read.

Related: the storage-side counterpart to the fast-lookup index (retrieval-side) and /obs-tokenguard (live command-cost side).

11. /obs-personality Identity & Build DNA

Stores a full identity/credential profile (git identity, tokens, API keys, associated projects) in the vault, keyed by persona name — the Obsidian-side counterpart to /personality.

What it does

Each persona is its own note under `Personalities/<persona-name>.md` — identity fields (git name/email), tokens/credentials (GitHub, Vercel, Supabase, Render, other API keys), and associated projects. Calling it with just a persona name reads and reports that profile (masking secret-looking values in the chat reply while the file itself keeps the full value); calling it with `field=value` pairs upserts those fields without dropping ones not mentioned in that call.

> **Standing risk, deliberately accepted:** this skill stores raw secret values in plaintext in the vault by explicit user choice, diverging from this environment's own separate reference-only credential pattern (`.secrets/<name>.env` plus a mode-600 file, used elsewhere). The user was shown that alternative directly and chose raw storage in Obsidian anyway. This means every credential here is only as safe as the vault itself.

Usage

Argument	Effect
`<persona-name>`	Reads and reports that profile (masked in chat).
`<persona-name> field=value ...`	Creates or updates fields on that profile.

Example

```
/obs-personality vedant github-token=ghp_xxxxxxxxxxxxxxxxxxx
```

Upserts the GitHub token field on `Personalities/vedant.md`, confirming back which field names were set — never echoing the full value in chat unless explicitly asked.

Related: the "who you are" counterpart to /obs-code-personality's "how you build."

12. /obs-code-personality Identity & Build DNA

Captures and applies your BUILD DNA — design style/colour palettes/web-dev structure, pentest ideology + preferred tools + HackerOne/Bugcrowd report style, and reusable code templates — so Claude builds and tests things your way without you re-explaining each time.

What it does

Build DNA is split into five domain notes under `CodePersonality/`: `design.md` (colour palettes with exact hex values, typography, layout preferences, things to always avoid), `webdev.md` (preferred stack, project structure, naming conventions, patterns embraced or rejected), `pentest.md` (testing ideology, the tools actually relied on and in what order), `reports.md` (HackerOne/Bugcrowd report structure, tone, severity framing, templates), and `code-templates.md` (reusable scaffolds to reuse verbatim).

It has two modes: recording a preference (routed to the right domain note, appended with dated provenance, never silently overwriting a contradicting prior preference — both are kept and the user is asked which is current), and applying build DNA before doing web-dev or pentest work (reading the relevant domain notes first and actually building to them — asking rather than guessing for anything still marked "not yet specified").

> **Honest limit:** this profile only grows from what the user actually provides. It never invents a palette, a stack preference, or a methodology — fields not yet specified are marked as such, and the skill asks instead of guessing.

Usage

Argument	Effect
`<a preference>`	Records it to the matching domain note.
`apply design|webdev|pentest|reports`	Loads that domain before building.

Example

```
/obs-code-personality background should be #F4F1EA, accent #B5652E, serif display type
```

Recorded to `CodePersonality/design.md` with dated provenance; a later `apply design` loads this before any new frontend work begins.

Related: the "how you build" counterpart to /obs-personality's "who you are." Feeds /obs-optimiser indirectly via recorded pentest tool preferences.

13. /obs-life Identity & Build DNA

Life & career journal — captures updates about YOU (role changes, priorities, constraints) and specifically WHY your working style shifts, so Claude understands the reason behind changes, not just the change.

What it does

Distinct from `/obs-closeday` (work progress): this captures life context that reshapes how the user works — a role change, a new job, a shift in priorities, a constraint. It records the update in the user's own words without editorializing, then adds an explicit "why this might change how I work" hypothesis (framed as something to confirm with the user, never asserted as fact) — this is what later lets Claude understand why a build-DNA or project-approach change happened, instead of treating a style shift as unexplained.

Each entry is its own dated note under `Life/<date>-<slug>.md` — deliberately, again, so `/trace <life theme>` can show how a situation evolved over time.

Usage

Argument	Effect
`<a life/career update>`	Required.

Example

```
/obs-life joining EY as Sr Security Engineer next month
```

Files a dated note with an "Impact on how I work" section: a hypothesis that enterprise-scope, report-heavy engagements and stricter authorization boundaries may shift pentest working style — flagged for the user to confirm, not stated as settled.

Related: feeds /obs-code-personality when a life change plausibly affects build/pentest preferences.

14. /obs-optimiser Automation Decision-Making

The decision agent for EXTERNAL tooling. Recommends the best tool/framework for a job from what's actually been recorded, logs the choice and its outcome, and improves its recommendations over time.

What it does

Deliberately distinct from `/obs-organiser` — that routes obs *skills*; this decides which *external* tool or framework to use (pentest automation, learning tools, automation frameworks). It has two modes: recommending (reading the relevant category log and any `/obs-n8n`, `/obs-crewai`, `/obs-hermes` notes bearing on the category, then naming a tool with the concrete evidence behind it — or saying plainly "not enough recorded to recommend confidently" rather than manufacturing a winner) and recording an outcome (after a tool was actually used, logging what happened so future recommendations improve).

> **Honest limit, stated in the skill itself:** it recommends only from evidence. If the recorded notes don't support naming a best tool, it says so and proposes gathering data first — it does not invent a "best tool" the notes don't support. This is the guardrail that keeps it a decision agent, not a hallucination engine.

Usage

Argument	Effect
`recommend <task category>`	e.g. "recommend pentest automation".
`record <tool> <outcome>`	Logs a real outcome.

Example

```
/obs-optimiser recommend pentest automation
```

Reads `Optimiser/pentest-automation.md` plus related n8n/CrewAI/Hermes notes; if two options are close, names the deciding factor rather than picking silently.

Related: fed by /obs-n8n, /obs-crewai, /obs-hermes.

15. /obs-n8n Automation Decision-Making

Captures your n8n knowledge — working workflows, node configs, credentials-handling patterns, gotchas, and what you automated — as dated notes that feed /obs-optimiser's automation recommendations.

What it does

Records a reusable workflow, a node/credential pattern, a gotcha, or an outcome as its own dated note under `Automation/n8n/`. Never stores real credentials or webhook secrets — it records the pattern ("uses an HTTP Request node with a bearer token from an n8n credential") and where the secret actually lives, scrubbing any pasted workflow JSON to placeholders and saying so.

Usage

Argument	Effect
`<a workflow/config/learning>`	Records it.
`list`	Shows what's already captured.

Example

```
/obs-n8n webhook -> HTTP Request (bearer from n8n credential) -> Slack notify;
took 3 tries to get the retry/backoff right on the HTTP node
```

Related: feeds /obs-optimiser's automation-framework recommendations, alongside /obs-crewai and /obs-hermes.

16. /obs-crewai Automation Decision-Making

Captures your CrewAI knowledge — agent/crew/task designs, tool wiring, prompt patterns, what worked and what didn't — as dated notes that feed /obs-optimiser's automation recommendations.

What it does

Same shape as `/obs-n8n`, scoped to CrewAI: crew/agent/task structures worth reusing, how tools were wired to agents, prompting patterns, and outcomes. Never stores real API keys — records which env var/provider was used, scrubbing pasted secrets to placeholders.

Usage

Argument	Effect
`<a crew/agent/task design or learning>`	Records it.
`list`	Shows what's already captured.

Example

```
/obs-crewai researcher + writer crew with a shared search tool cut token
usage ~30% vs. a single monolithic agent on the same task
```

Related: feeds /obs-optimiser, alongside /obs-n8n and /obs-hermes.

17. /obs-hermes Automation Decision-Making

Captures your Hermes knowledge — configs, workflows, and outcomes — as dated notes that feed /obs-optimiser. On first substantive use it confirms WHICH Hermes you mean so notes stay accurate.

What it does

Same shape as /obs-n8n and /obs-crewai, with one deliberate extra step: "Hermes" is an ambiguous name (it could mean the Nous Research Hermes LLM family, a messaging/automation framework, or something specific to the user's own stack). On the first substantive note — or if the index hub doesn't yet record which one — the skill asks the user directly which Hermes is meant and records the answer at the top of the index, rather than guessing. A wrong assumption here would poison every downstream recommendation, so this disambiguation is treated as mandatory, not optional.

Usage

Argument	Effect
`<a config/workflow/learning>`	Records it (disambiguates first, if needed).
`list`	Shows what's already captured.

Example

```
/obs-hermes [first use] -> Claude asks which Hermes, user answers,
then: config X worked well for Y
```

Related: feeds /obs-optimiser, alongside /obs-n8n and /obs-crewai.

18. /obs-list Discovery & Connections

Detailed inventory of everything in the vault's /obs- system — built from the fast filesystem index, not a full vault scan — and wired into /obs-connect, /obs-learn (+ /obs-learn-cyber, /obs-mistakes), and /trace.*

What it does

Enumerates every label directory under `.obs-index/by-label/` (a filesystem listing, not a search) and organizes the results into sections matching the established folders — Journal, Learnings, Mistakes, CyberAI, Personalities, Pentest, Connections, Context log — each entry shown with date, time, a wikilink, and a content-derived gist. It then does three things no other skill does: surfaces cross-section pairs worth running `/obs-connect` on (without duplicating that analysis itself), flags any label/topic with two or more entries as "traceable," and collects those into a closing "Ready to /trace" list with the exact commands to run.

Usage

Argument	Effect
`(none)`	Full inventory.
`<label/kind>`	Filters to that label only, e.g. "pentest" or "cyberlearn".

Example

A real run of this skill against this project's own vault (2026-07-05) reported: Personalities populated (three bootstrap-imported personas), every other section "none found" (most of the system was built but not yet exercised), and "none found" for both cross-connections and traceable topics — an honest snapshot rather than a padded report.

Honest limit: the fast index is an accelerator; when it doesn't yet cover something (a note pre-dating the index, or written by something other than an obs-* command), the skill falls back to a full search rather than silently missing it.

Related: reads the index built by every other skill; feeds /obs-connect and /trace candidates.

19. /obs-connect Discovery & Connections

Bridges two topics using your vault's link graph and shared tags to surface connections you haven't noticed yet.

What it does

Given two topics, it independently searches for notes under each, builds a link graph (outbound wikilinks and backlinks for every candidate note), and looks for three kinds of connection: a direct link between a topic-A note and a topic-B note, a bridge note that links to both but belongs to neither, and shared tags across the two sets. It reports direct links, bridge notes, shared tags, and a short "pattern observed" — or says plainly that no connection was found, rather than forcing one that isn't there.

Usage

Argument	Effect
`<topic A> | <topic B>`	The two topics to bridge (also accepts "and"/"vs" as separators).

Example

```
/obs-connect vedant persona | excelonCS pentest engagement
```

Auto-saves the result to `Connections/<topic A>--<topic B>.md` (overwriting on rerun), with the note itself wikilinking every A-note and B-note used in the analysis — the note is the bridge, not just a description of one.

Related: candidates for this skill are surfaced by /obs-list; often triggered by the PostToolUse hook when a new vault note links across sections.

20. /trace Discovery & Connections

Trace how an idea/topic has evolved over time across the Obsidian vault — first appearance, evolution, and current connections.

What it does

The original vault-history skill, predating the rest of the family's linking/index protocol conventions. Searches broadly for every mention of a topic (vault search plus a direct grep, since each catches things the other misses), approximates backlinks (since there is no dedicated backlinks API) by grepping for each candidate note's title, and recurses one level to catch related-by-theme notes sharing tags. It then establishes chronology from frontmatter dates or, failing that, filesystem timestamps (marked approximate), and reports first appearance, evolution (skipping notes that just repeat an earlier mention with nothing new), current connections, and — only when the source notes are genuinely cluttered or repetitive — an organized synthesis that consolidates without inventing content.

Usage

Argument	Effect
`<topic>`	Required — the skill asks and stops if empty.

Example

```
/trace MCP server security
```

If `CyberAI/` holds two or more notes about MCP security (filed by `/obs-learn-cyber`), this produces a genuine timeline of how understanding developed, not just a single-note summary.

Auto-saves the timeline to `Traces/<topic>.md` — overwriting on rerun, since a new run supersedes the old one rather than piling up duplicates.

Related: this is the payoff skill for every capture skill that deliberately files one note per event instead of one growing log — /obs-learn-cyber, /obs-life, /obs-requests all exist in that shape specifically so /trace has real material to work with.

21. /obs-pentest Security

Full recon→enum→exploit→report bug-bounty pentest agent (self-contained duplicate of /pentest), with continuous Obsidian vault logging of POCs and workflow so progress and learnings persist across sessions.

What it does

A complete, standalone pentest pipeline — scope enforcement first (reads `scope.md`, checks every target against it before any tool runs; a blanket "everything is legal" statement from the user is noted but never substitutes for an actual scope file), then recon, enumeration, a full OWASP Top 10 + WSTG-based vulnerability-testing pass, real-exploit-only exploit development (with a reverse-shell reference used strictly after RCE is confirmed on an in-scope target), and a HackerOne/Bugcrowd-standard report structure. What makes the "obs" version distinct from the plain `/pentest` command it duplicates is continuous vault logging: after every phase (not just at the end), a timestamped entry is appended to `Pentest/<target>/pen-context.md`, making the engagement resumable across sessions without re-reading every raw tool output again. The finished report also mirrors into the vault, and lessons cross-write to `~/.claude/knowledge/pentest-learning.md`, which is read at the start of every future engagement.

The zero-hallucination protocol

Every finding must trace to a verbatim line from a tool's output — no evidence, no finding. Generic ("may be vulnerable") claims without tool confirmation go under Potential Findings, explicitly labeled UNCONFIRMED. CVSS scores require a citable CWE/CVE, otherwise "CVSS: TBD." Under-reporting is treated as acceptable; a fabricated finding is not — a missed vulnerability is survivable, a fake one submitted to a bounty platform is not.

Usage

Argument	Effect
`<path to scope.md>`	Required — the skill stops and asks if no scope.md is found.

Example

```
/obs-pentest /home/kali/Desktop/airbnb/scope.md
```

Creates both a local job directory and a vault note (`Pentest/airbnb/pen-context.md`), runs the full pipeline with continuous vault updates, and produces a critical-first report saved to both locations.

Related: the organiser routes this to "ask first" — active tooling against a target always requires confirmation, never auto-triggered.

22. /obs-tokenguard Efficiency

Reviews this session's tool calls for token cost, flags the expensive ones, and suggests a cheaper approach for next time. Root-scoped. Pairs with a real-time PreToolUse nudge on large Reads.

What it does

Two real mechanisms work together. First, a genuine `PreToolUse` hook on the `Read` tool (a real shell script that `stat`s the target file's size, not a static message) fires before any read over roughly 50KB, naming the file and its size and suggesting a cheaper approach — a targeted `Grep` if searching for something specific, `Read` with `offset` / `limit` if only a section is needed, or running `/obs-distil` afterward if it's a large reference document being captured. It never blocks the read — a genuinely necessary full read always proceeds; this is a reminder delivered at the moment it matters, not a gate. This hook was verified firing live against a real 288KB file before being wired into the configuration.

Second, the skill itself reviews the session on demand (or near session end): the largest/most expensive calls made, what a cheaper approach would have looked like, and whether a repeated pattern is worth adopting as a standing default — offered to `/obs-code-personality` rather than recorded automatically, since that skill only grows from explicit input.

> **Honest limit:** there is no exact per-call token meter exposed to a skill or hook. Every cost figure here is an approximation (roughly, output characters ÷ 4), reported as "approximately" — never as an exact number.

Usage

Argument	Effect
`(none)`	Reviews the current session.

Example

```
/obs-tokenguard
```

Might report: "Read of a 300-line config file to find one setting — approximately 2,000 tokens; a targeted Grep would have cost a few hundred" — and ask whether that pattern (reading whole config files) is common enough to record as a standing preference.

Related: the live command-cost counterpart to /obs-distil (storage-side) and the fast-lookup index (retrieval-side).

23. /obs-requests Efficiency

Fires just before a session ends — reviews this session's prompts, researches the ambiguous/typo-heavy/ misrouted ones, and drafts a clean "what you meant" version of each. Next time a similar prompt comes in, Claude checks this first, proposes that refined interpretation, and confirms before proceeding.

What it does

At session end, this skill looks back over the session's own prompts (only what's still in context — anything already lost to compaction isn't recoverable here) and picks out the ones that needed real work to understand: typos, ambiguity, a clarifying question that had to be asked, a correction or redirect after acting, or a case where content was dropped into the wrong command entirely. For each, it researches what the user actually meant — using the clarifying questions asked, corrections given, and the final accepted outcome as evidence — and writes a refined version: the same request, restated clearly and unambiguously, structured the way it should have been phrased to get the right result on the first pass. The original is kept completely verbatim, typos included, since that raw phrasing is the match signature used later.

The real payoff is what happens on a future similar prompt. Rather than a long re-confirmation, the skill states the inferred requirement plainly and confidently — the whole point of accumulating history is that Claude should increasingly know what the user means without re-deriving it from scratch — then asks a fast permission check ("proceeding on that basis — go ahead?") rather than a drawn-out re-explanation. The confidence is meant to visibly grow as more matching history accumulates: a topic with one prior entry still deserves a fuller check; a topic with many consistent entries deserves a short one. Matching itself is best-effort (keyword/topic overlap via the index, not guaranteed semantic matching) — which is exactly why the permission check happens every time a match is used, even a fast one. Fast is never the same as skipped.

Usage

Argument	Effect
`(none)`	Fires via the SessionEnd hook.
`review <topic>`	Checks filed refined prompts for a topic on demand.

Example

A prompt this session got redirected into the wrong command's argument by accident. At session end, `/obs-requests` files the original (verbatim, typos included), the evidence of what was actually meant, and a clean refined version under `Requests/2026-07-06-<slug>.md`. The next time a similarly-shaped prompt arrives, the organiser states "this looks like <refined interpretation> — proceeding on that basis, go ahead?" instead of re-deriving the same misunderstanding.

Related: fires via the same `SessionEnd` hook as /obs-closeday. Feeds the organiser's routing confidence over time.